

Design Document - Codity Distributed Job Scheduler

1. System Architecture

Overview

Codity is designed as a distributed job scheduling system with clear separation of concerns between API handling, job execution, and user interface. The system follows a shared-database architecture where the PostgreSQL database serves as both the job queue and the coordination mechanism between services.

Architecture Decision: Shared Database vs Message Broker

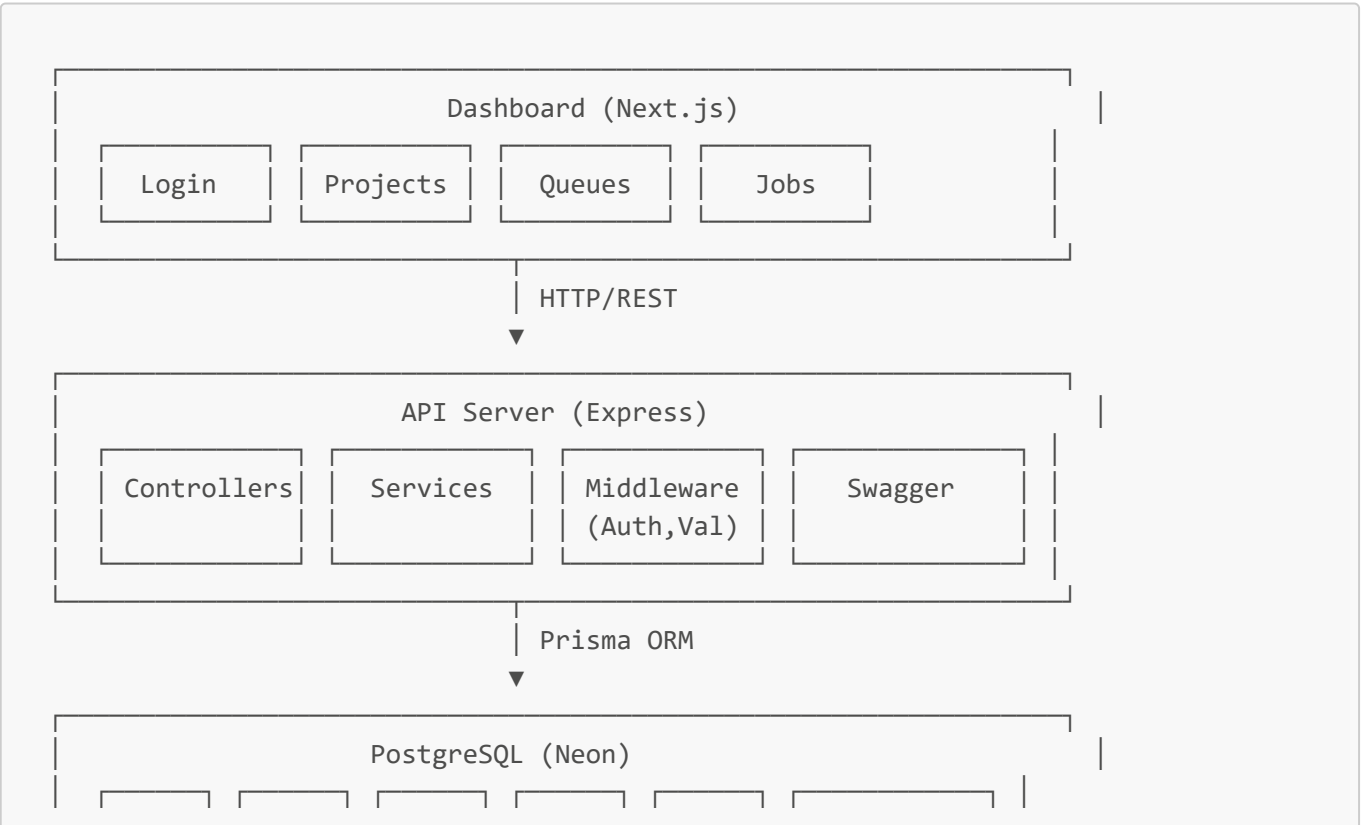
We chose a database-backed queue over a dedicated message broker (Redis, RabbitMQ, Kafka) for several reasons:

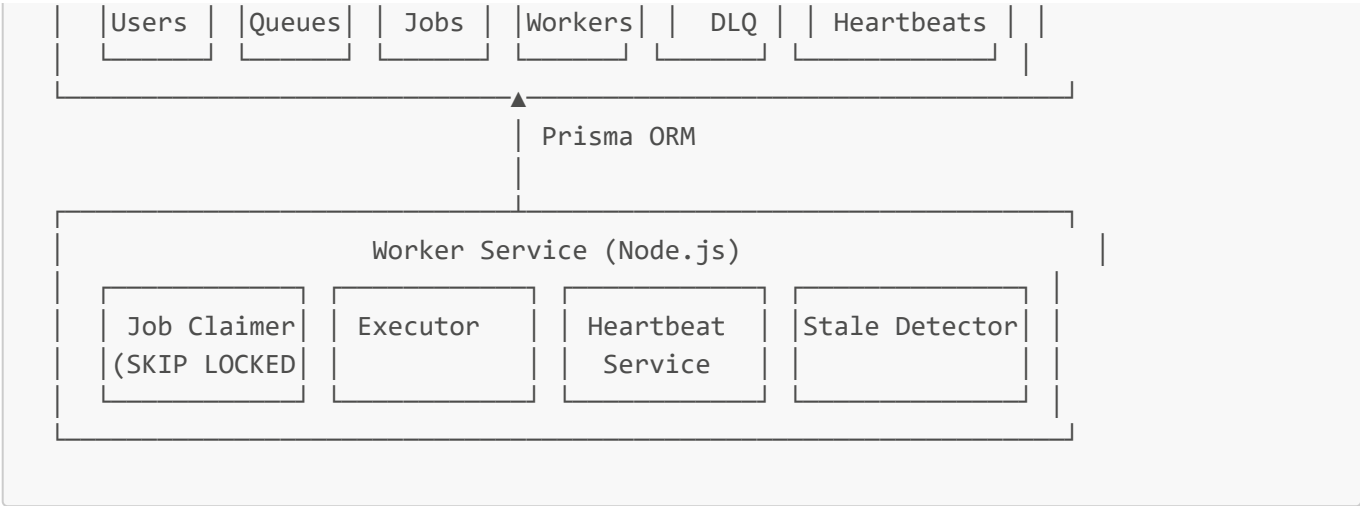
- 1. **Operational simplicity:** One fewer infrastructure component to manage
- 2. **Transactional guarantees:** Job state changes are ACID-compliant
- 3. **Query flexibility:** Complex filtering, pagination, and analytics on job data
- 4. **Durability by default:** No need for separate persistence configuration
- 5. **PostgreSQL advisory locks and SKIP LOCKED:** Provides the concurrency primitives we need

Trade-offs accepted:

- Slightly higher latency compared to in-memory brokers (~5-50ms vs ~1ms)
- Polling-based instead of push-based (acceptable at moderate scale)
- Write amplification from status updates

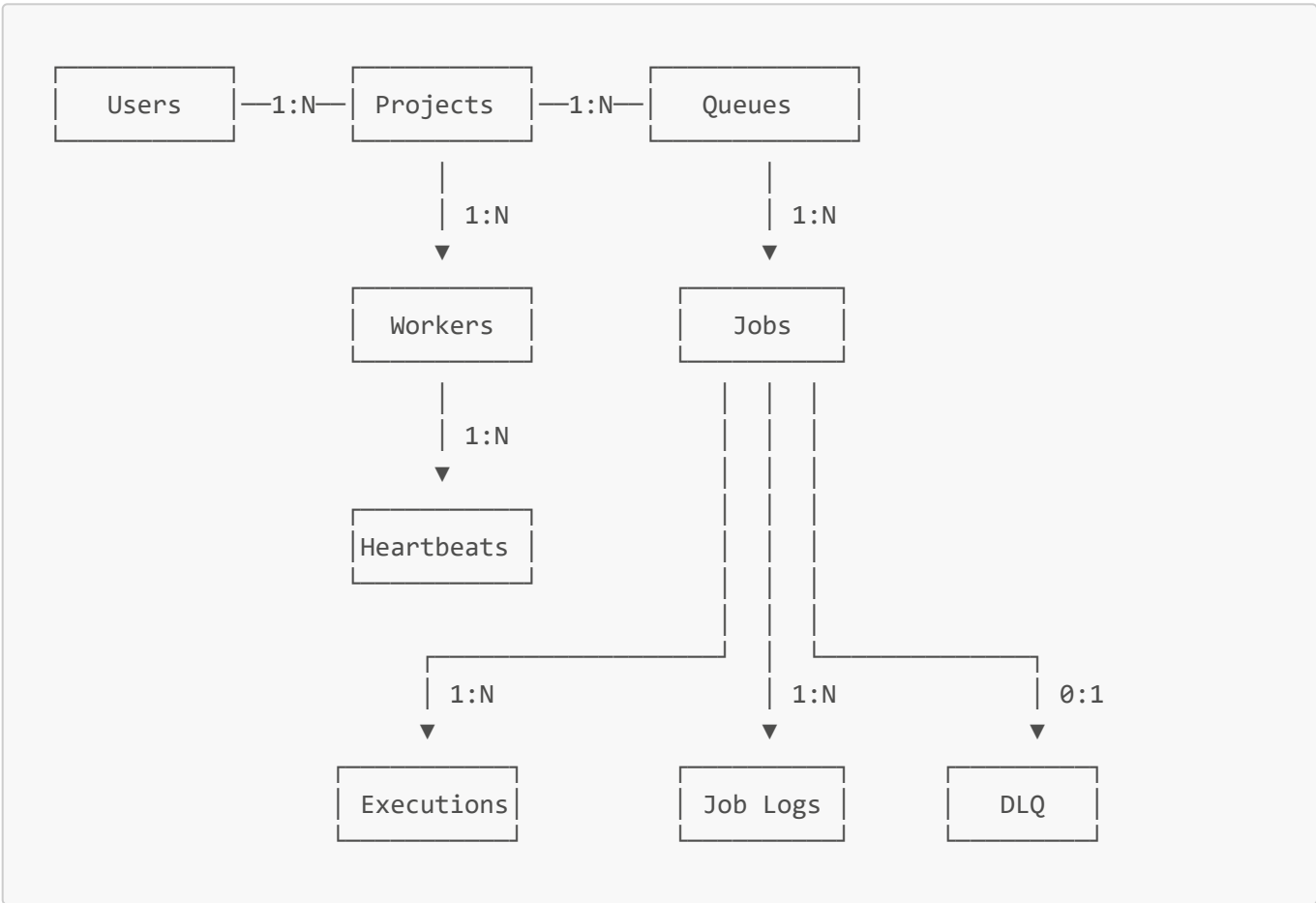
Component Interaction





2. Database Design

Entity Relationship Diagram



Key Design Decisions

UUID Primary Keys

UUIDs allow ID generation at the application layer without database coordination. This is essential for distributed systems where multiple workers may be creating records simultaneously.

Composite Index Strategy

The critical index for job claiming is:

```
(queue_id, status, priority, scheduled_for)
```

This covers the exact query pattern workers use when polling for available jobs, enabling index-only scans.

Separated Heartbeat Table

Worker heartbeats are high-frequency writes (every 10s per worker). Storing them in a separate table prevents write amplification on the workers table, which is read-heavy for dashboard queries.

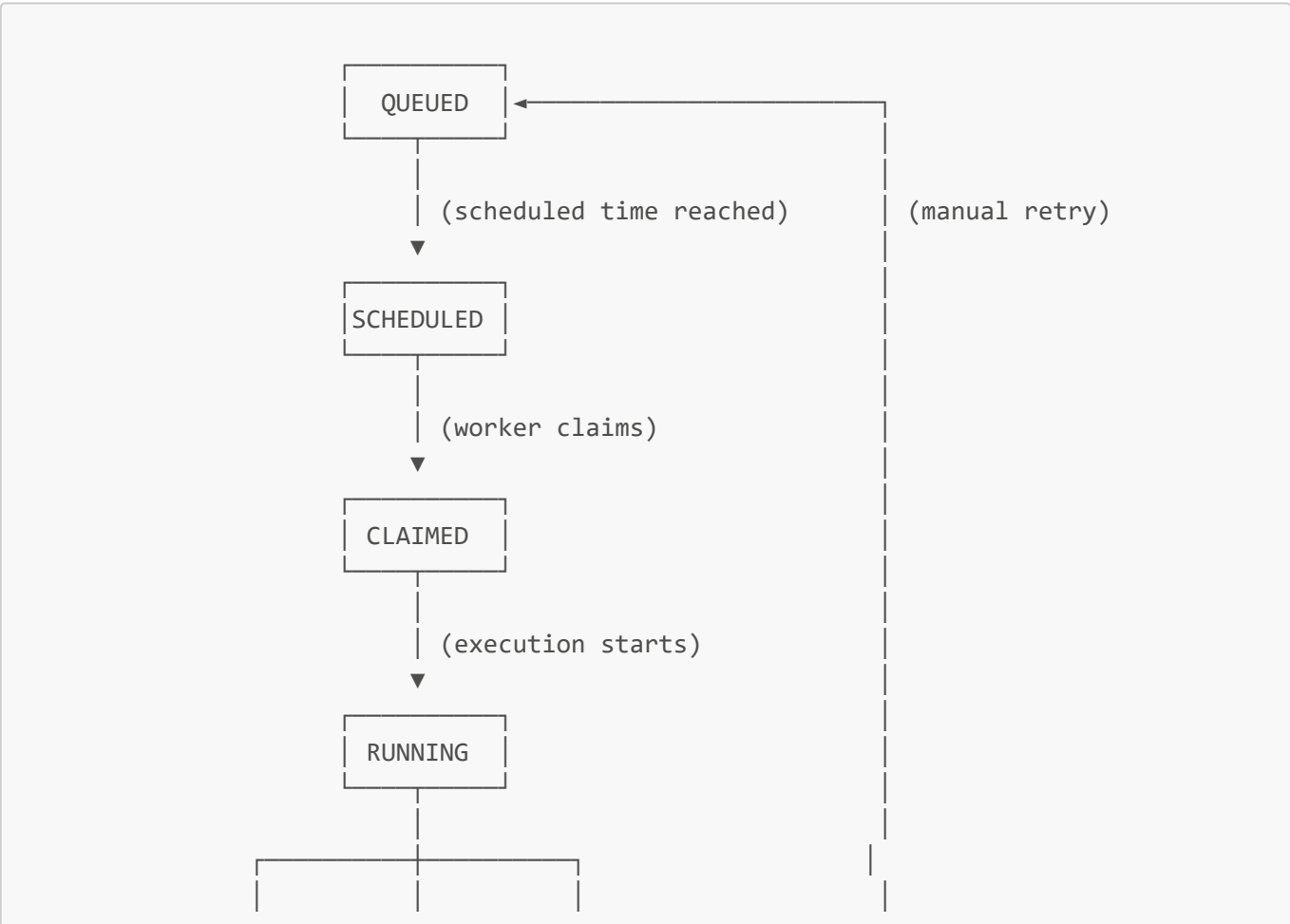
Dead Letter Queue as First-Class Entity

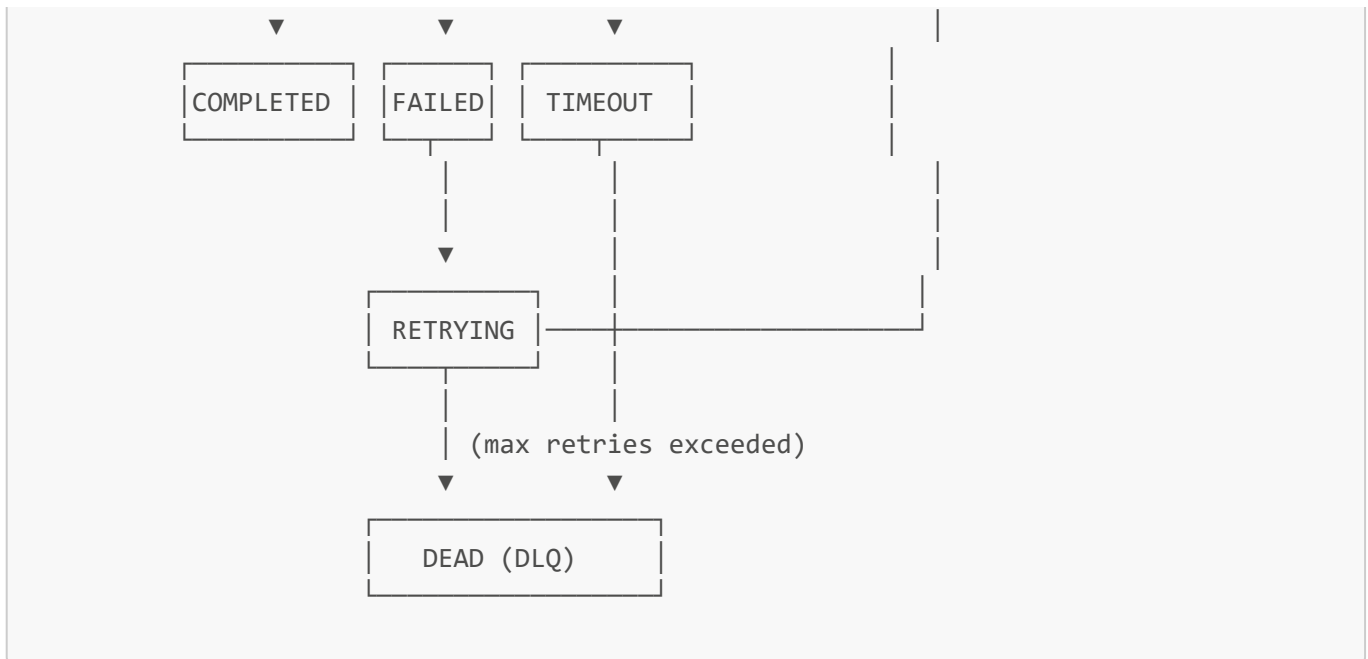
Rather than just marking jobs as "permanently failed," we create explicit DLQ entries with metadata. This enables:

- Separate monitoring and alerting
- Bulk retry operations
- Root cause analysis
- Resolution tracking

3. Job Lifecycle

State Machine





4. Concurrency & Reliability

Atomic Job Claiming

The core reliability primitive is PostgreSQL's `FOR UPDATE SKIP LOCKED`:

```

UPDATE jobs
SET status = 'CLAIMED', claimed_by = $worker_id, locked_until = $lock_ttl
WHERE id IN (
  SELECT id FROM jobs
  WHERE status IN ('QUEUED', 'RETRYING')
    AND queue_id IN (SELECT id FROM queues WHERE status = 'ACTIVE')
    AND (scheduled_for IS NULL OR scheduled_for <= NOW())
  ORDER BY priority DESC, created_at ASC
  LIMIT $batch_size
  FOR UPDATE SKIP LOCKED
)
RETURNING id
  
```

Why this works:

- `FOR UPDATE` acquires row-level locks on selected jobs
- `SKIP LOCKED` causes concurrent workers to skip already-locked rows
- Combined effect: exactly-once claiming without application-level coordination
- No race conditions, no duplicate processing

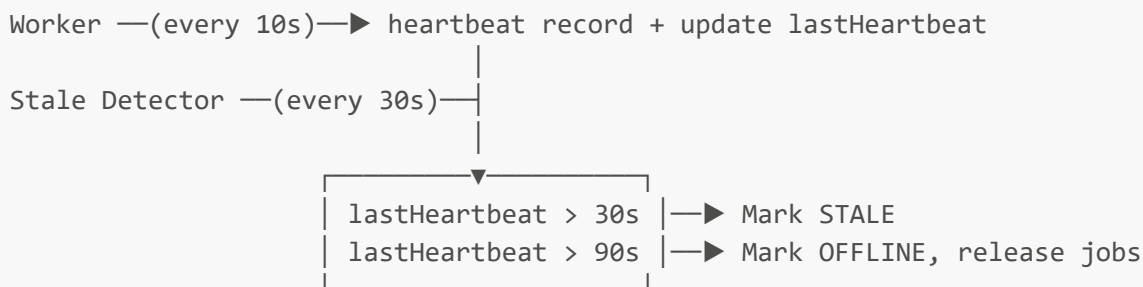
Lock TTL (Time-To-Live)

Each claimed job has a `locked_until` timestamp. If a worker crashes:

1. The lock TTL expires
2. The stale detector marks the worker OFFLINE

3. Jobs are released back to QUEUED status
4. Another worker can claim them

Worker Heartbeats



5. Retry Strategies

Fixed Delay

Attempt 1 —[5s]—> Attempt 2 —[5s]—> Attempt 3

Use case: External services with fixed rate limits.

Linear Backoff

Attempt 1 —[5s]—> Attempt 2 —[10s]—> Attempt 3 —[15s]—> ...

Use case: Gradually increasing pressure relief.

Exponential Backoff with Jitter

Attempt 1 —[1s+jitter]—> Attempt 2 —[2s+jitter]—> Attempt 3 —[4s+jitter]—> ...

Use case: Default strategy. Prevents thundering herd on recovery.

Jitter adds random variance (up to 30% of the delay) to prevent multiple retrying jobs from all hitting the same resource simultaneously.

6. Scalability Considerations

Horizontal Scaling

- **API Servers:** Stateless, scale behind a load balancer
- **Workers:** Each registers independently, claims jobs independently

- **Database:** Connection pooling via PgBouncer/Neon's built-in pooler

Performance Bottlenecks and Mitigations

Bottleneck	Mitigation
Job claiming contention	SKIP LOCKED eliminates lock waiting
Heartbeat write volume	Separate table, periodic pruning
Job table growth	Archival strategy for completed jobs
Dashboard query latency	Materialized stats, caching layer

7. Security Model

- JWT-based authentication with configurable expiry
- Project-level isolation (users can only access their own projects)
- Rate limiting on API endpoints (100 req/15min window)
- Helmet.js for HTTP security headers
- Input validation via Zod schemas
- bcrypt (cost factor 12) for password hashing

8. Observability

- Structured JSON logging (Winston)
- Per-job execution logs with levels (DEBUG, INFO, WARN, ERROR)
- Worker metrics (processed count, failure count, memory, load)
- Job execution duration tracking
- Dead letter queue monitoring